Low-Cost FPGA-Based Bit Error Rate Test set

Vishnu Barath

3rd Year Project Interim Report

Department of Electronic &
Electrical Engineering

UCL

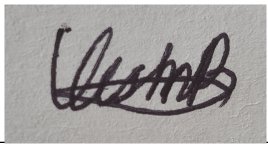
Supervisor: Martyn Fice

December 9, 2025

I have read and understood UCL's and the Department's statements and guidelines concerning plagiarism.

I declare that all material described in this report is my own work except where explicitly and individually indicated in the text. This includes ideas described in the text, figures and computer programs.

This report contains 17 pages (excluding this page and the appendices) and 2496 words.

Signed:  Date: December 9, 2025

(Student)

Low-Cost FPGA-Based Bit Error Rate Test set

Vishnu Barath

This project aims to build a low-cost system that can test the Bit Error Ratio of a communication channel. It will use Pseudorandom Binary Sequence generators to generate a deterministic bitstream which will be transmitted across the channel. The sequence will be generated on both the transmitter and the receiver to compare and calculate the BER. So far, a transmitter has been developed which generates a repeating PRBS-7 sequence with a length of 127 bits. The PRBS generator is implemented using a Linear Feedback Shift Register controlled by a 10MHz clock where a bit is transferred at every rising edge, achieving a bitrate of 10Mbps. This has been implemented on an FPGA.

Contents

1 Project Description

Communication networks are continuously expanding in capacity and speed and being able to ensure signal integrity is paramount. The most common example of such a system is the internet, which uses undersea optical fibres for data transmission. Such networks have become a central component today with applications in many industries, such as telecommunications, cloud services, banking, healthcare and transportation. In accordance with these advancements, techniques to measure performance of communication channels have evolved. Such channels are susceptible to incorrect transmissions due factors such as noise or interference. A metric of performance for communication channels is the Bit Error Ratio (BER) which can be characterised via a Bit Error Rate Test (BERT). [1]

The BER of a channel is the proportion of erroneous bits in a transmission. When received data has errors it requires retransmission or extra processing from error correction. Hence, high BERs would significantly impact bitrate capability. [1] Being able to measure this is important in ensuring speed and efficiency in data transmission. Unfortunately, systems that measure BERs can be expensive and would be designed for bitrates in the Gbps range. Acquiring such equipment could prove difficult and excessive since high speeds aren't always required. [2]

This project aims to construct a system that can measure the BER of a Visible Light Communication (VLC) channel at bitrates up to 10Mbps, while keeping costs relatively low. Below are the main goals and objectives.

1. **Generate a Pseudorandom Binary Sequence (PRBS)**
   - **To test capability of the hardware choice**
   - **To set a fixed bit sequence length**
   - **To generate an apparently random, but known, bit sequence**
   - **To reach bitrates of up to 10Mbps**
2. **Develop an Error detection algorithm**
   - **To receive bitstream at up to 10Mbps**
   - **To generate PRBS on receiver side**
   - **To synchronise PRBS with the received bitstream**
   - **To calculate BER**

2 Literature Review

Building a system to measure a BER can be done via programmable devices such as Microcontrollers or Field Programmable Gate Arrays (FPGA). A sequence of known bits is required for testing a BER. One common method for generating one is the PRBS. Such sequences are generated using a Linear Feedback Shift Register (LFSR) where two specific bits in the register are inputs to an XOR gate then into the Least Significant Bit of the register. [3, 4] Low cost BERTs have been developed using such methods. [2, 5, 6] Examples of these are discussed below.

2.1 Serial Communication BERT

An FPGA-based device was developed to test the BER of a serial communication channel. It uses a 31-bit LFSR to generate a PRBS which would have a length of $2^{31} - 1$ **bits (**$\sim 2.14 \times 10^{9}$**). [2, 3]**
The system works by transmitting a PRBS to an external channel and receiving the bitstream on the same FPGA. The bits are generated, received and compared simultaneously. It's capable of testing bitrates of up to 50Mbps and used the Anritsu MP8302A BERT for reference. [2] By transmitting and receiving on the same FPGA, having to synchronise bitstreams wasn't an issue. However, this isn't good for simulating realistic transmission between two different devices.

2.2 VLC BERT

A PCB with logic circuits was designed to test a VLC channel, the same type of channel this project is meant for. In this case, the bit generator is a square wave rather than a PRBS. Effectively, it is a sequence of alternating ones and zeros. [5] Two separate PCBs transmit and receive the bitstreams. This BERT can test for bitrates up to 370kbps, which is significantly lower than this project's target. Using a periodic square wave simplifies the issue of synchronising the transmitted and received bitstreams. This system is significantly cheaper and less complex than FPGA equivalents. Unfortunately, bitstreams realistically won't be sequential ones and zeros, meaning the displayed BER may not reflect real world conditions for data transmission. [5]

2.3 Microcontroller-based BERT

An STM32 Microcontroller combined with a Complex Programmable Logic Device was used to make a BERT. A specific type of communication channel isn't specified. This system uses pseudorandom patterns to test a channel. [6]
A single STM32 transmits and receives the bitstreams and tests the BER at 2.048Mbps. [6] This bitrate is similar to what this project is targetting. Due to using only one device, this may not accurately simulate real world conditions.

2.4 Summary

Overall, both FPGAs and Microcontrollers are valid options in building a BERT since they can both be programmed to transmit and receive a PRBS and compare bitstreams. To best simulate realistic transmission, separate devices for transmitting and receiving would be ideal. While this may increase cost, relative to the price of BER equipment, it shouldn't be significant. [2, 5] A PRBS generator will be used to generate bits for calculating the BER. Figure 1 gives an overview of the system.

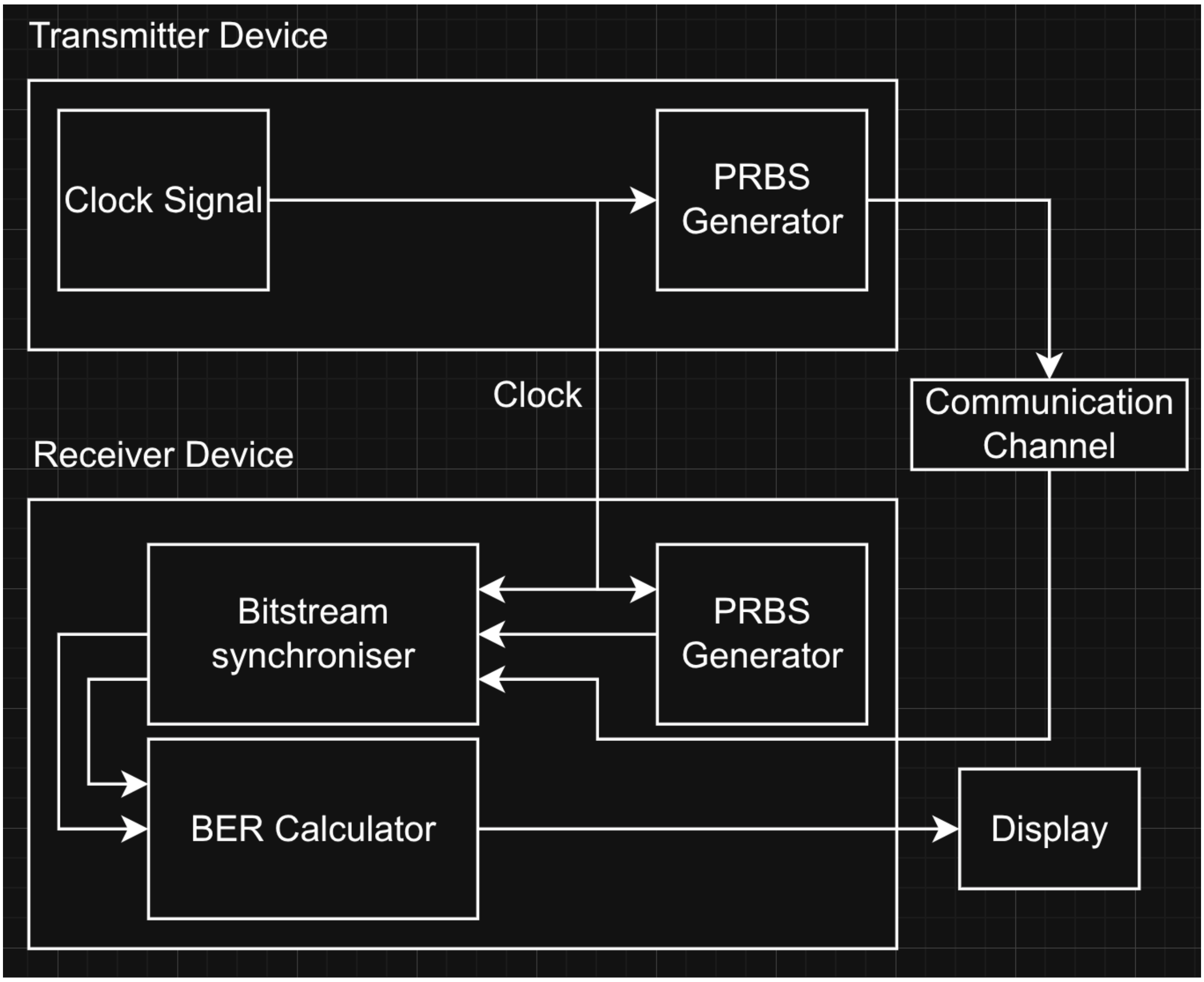


Figure 1: Block Diagram of BERT system

3 Work Performed to-date

3.1 Choice of Hardware

In this project, there is a choice between using Microcontrollers or FPGAs. To determine which is suitable, the capabilities of a device of each were tested using a 10MHz clock since this would allow bitrates of 10Mbps (bits are transferred on each rising edge).

3.1.1 MC

The board being tested here is the NUCLEO-F401RE which has STM32F401RET6 Microcontroller using the Serial Peripheral Interface (SPI) communication protocol and programmed using STM32CUBEIDE. [7, 8]

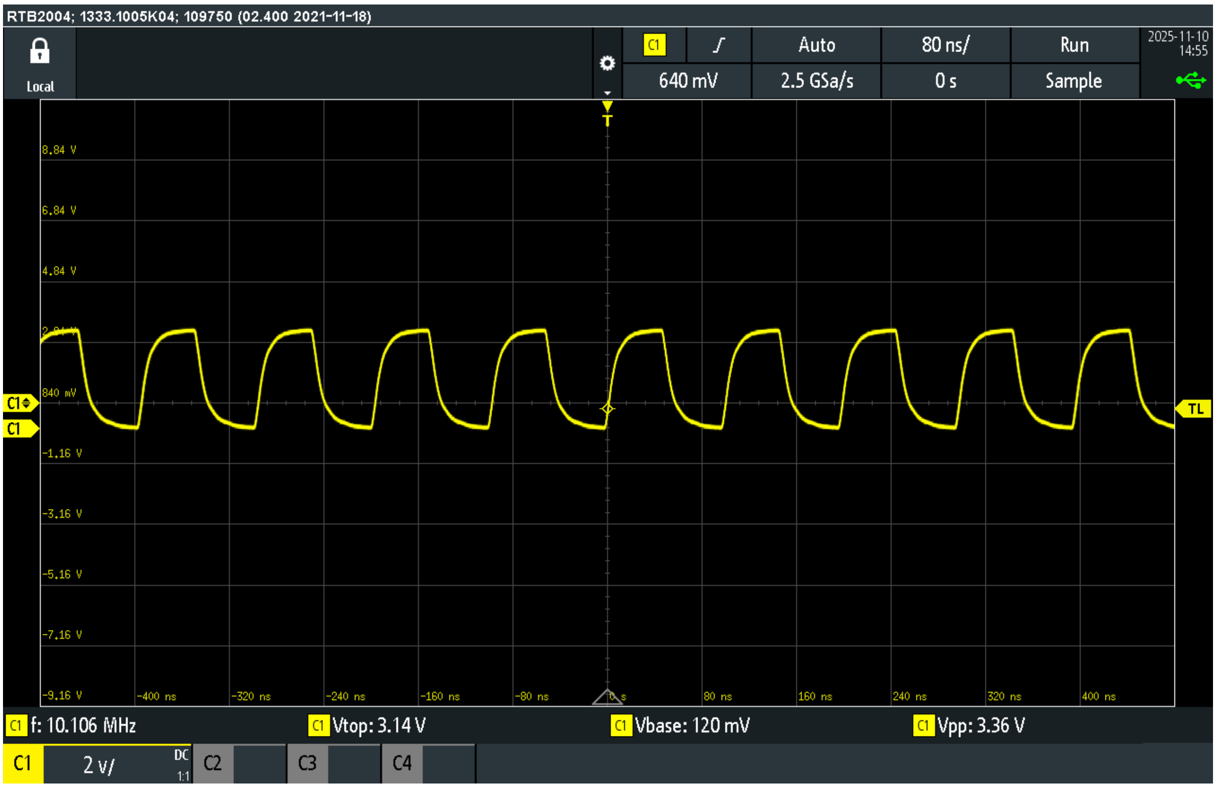


Figure 2: 10MHz clock from NUCLEO-F401RE (Oscilloscope capture)

In Figure 2, the clock is the expected frequency and peak-to-peak voltage (V_{pp}). However, the signal varies exponentially rather than instantaneously like a square wave. Different frequency clocks should be tested for this effect.

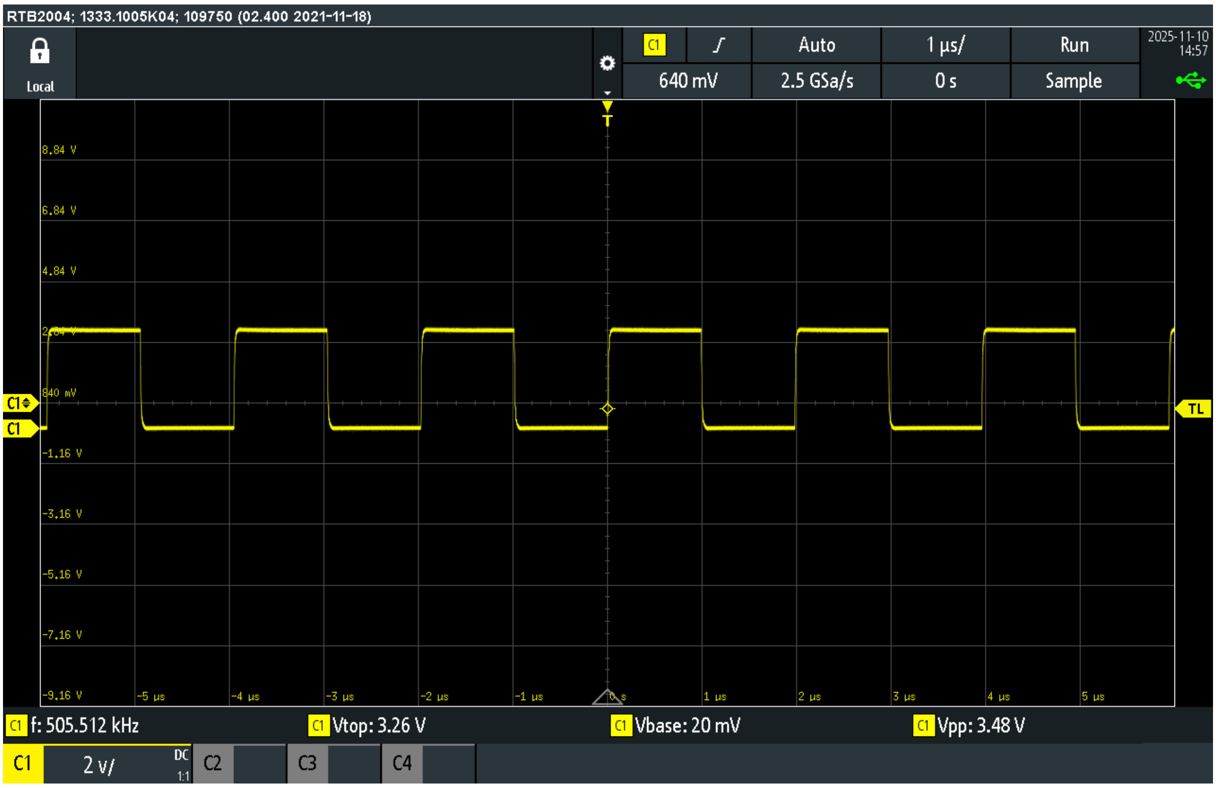


Figure 3: 500kHz clock from NUCLEO-F401RE (Oscilloscope capture)

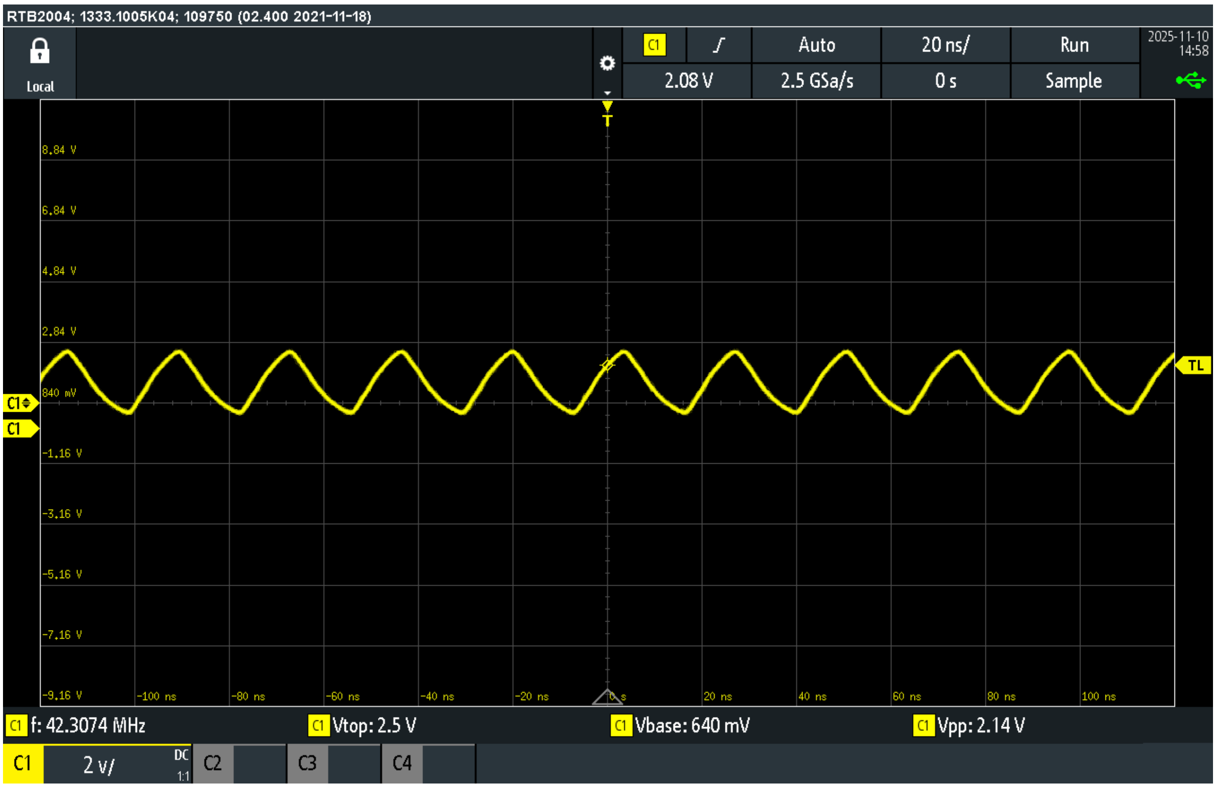


Figure 4: 42MHz clock from NUCLEO-F401RE (Oscilloscope capture)

Figure 3 shows the microcontroller producing a square wave clock at 500kHz, suggesting a measurement equipment limitation. In Figure 4, the microcontroller is producing a clock at its maximum frequency of 42MHz. [7, 8] The signal is almost triangular, but still holds a fundamental frequency of 42MHz. However, the V_{pp} is attenuated compared to that at 10MHz in Figure 2 and 500kHz in Figure 3.
The RT-ZPO3 oscilloscope probe being used has a -3dB bandwidth of 10MHz which explains the distortion in the 10MHz and 42MHz clocks. [9] The Fourier Series of a square wave, with its fundamental frequency, has higher frequencies that allow the wave to be square. [10] So in a 10MHz square wave, there are high frequencies which allow for the wave to be square. These frequencies are attenuated, giving the output in Figure 2. The 10MHz frequency is within the bandwidth, so there's little attenuation in V_{pp}, unlike at 42MHz in Figure 4. From testing at a lower frequency, it can be concluded that the hardware still produces a square wave at 10MHz with measurements being limited by the probe.

3.1.2 FPGA

The board being tested here is a DE0-Nano with an ALTERA Cyclone IV EP4CE22-F17C6 FPGA programmed in Quartus Prime Lite. [11]

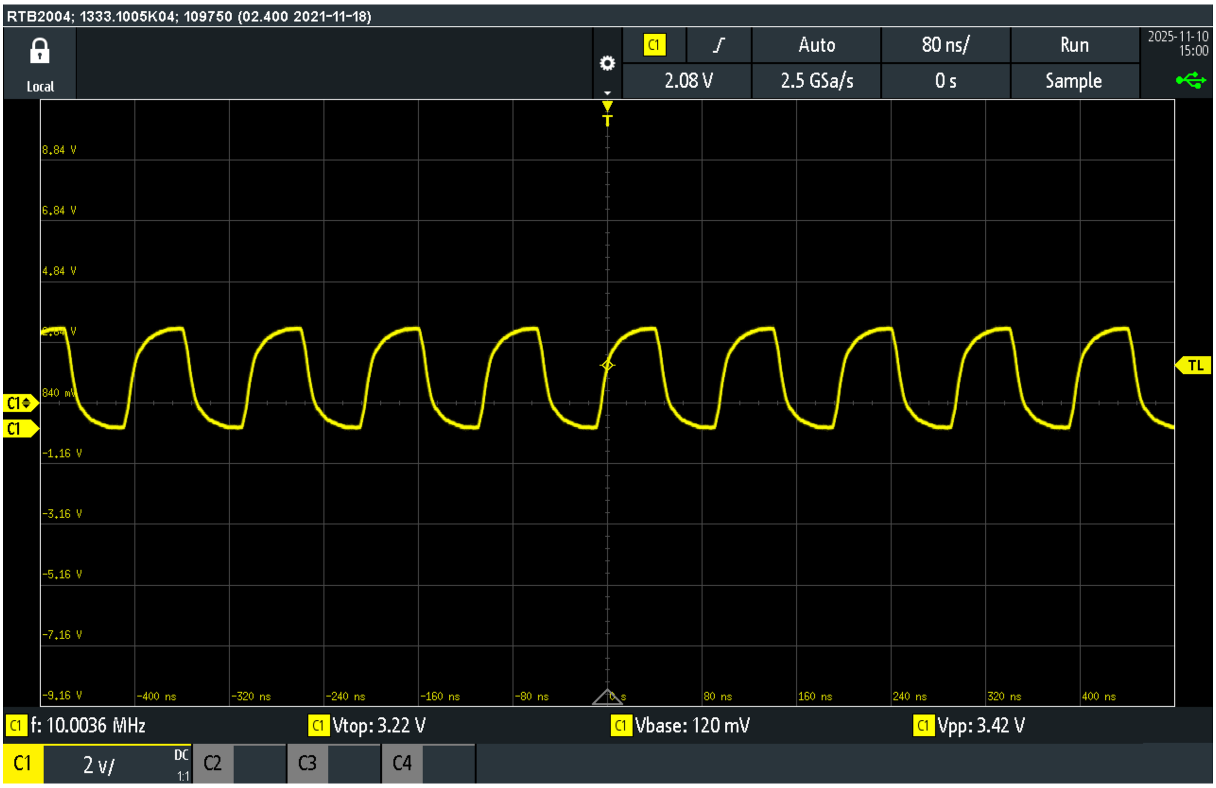


Figure 5: 10MHz clock from DE0-Nano (Oscilloscope capture)

Figure 5 shows that at a 10MHz clock, the FPGA provides a similar output to the microcontroller, with the probe causing similar distortion.

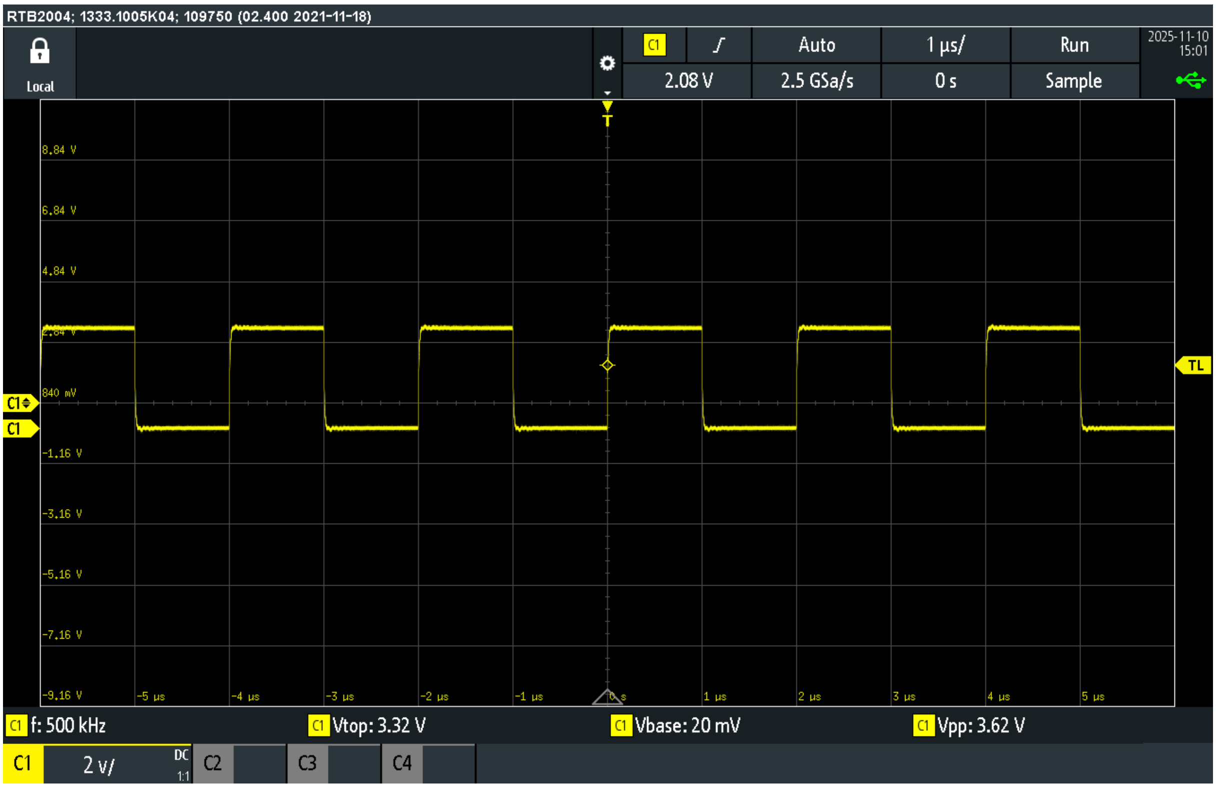


Figure 6: 500kHz clock from DE0-Nano (Oscilloscope capture)

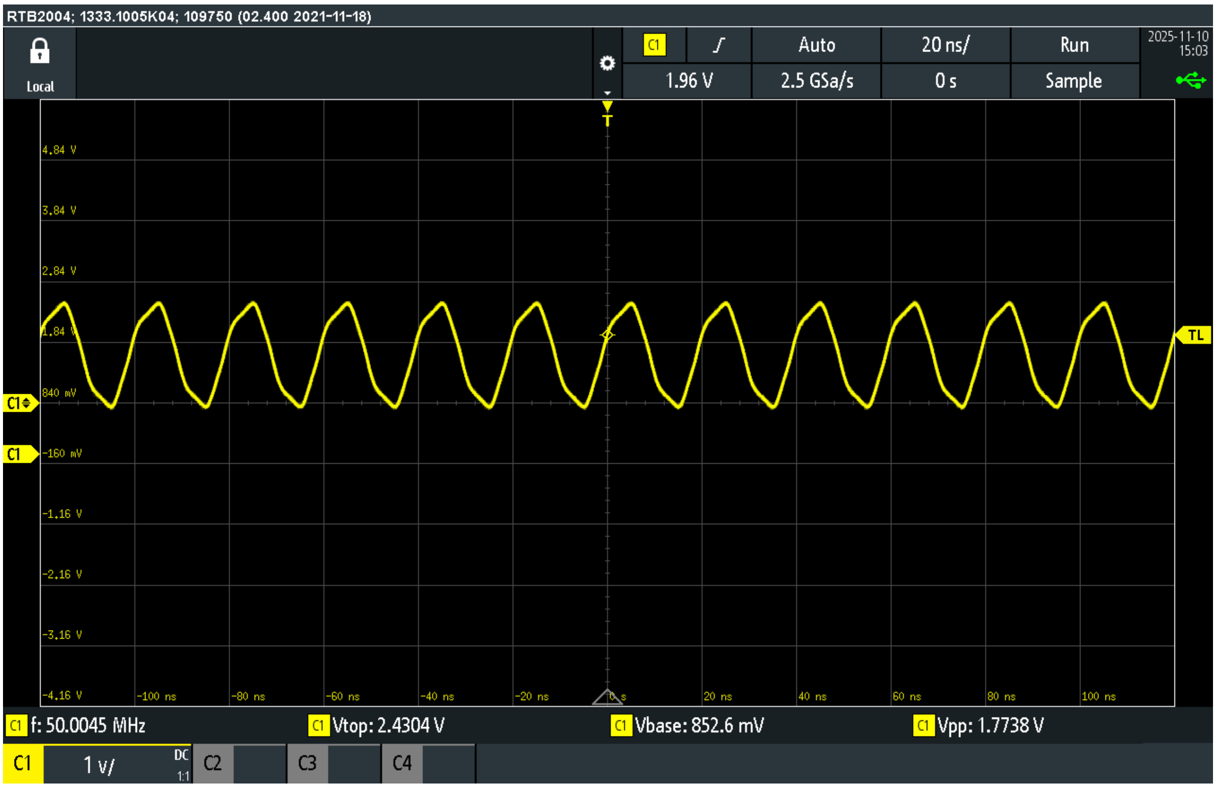


Figure 7: 50MHz clock from DE0-Nano (Oscilloscope capture)

When the clock is turned down to 500kHz, in Figure 6, like the microcontroller, the wave appears square due to the clock frequency being within the bandwidth of the probe. [9]
The FPGA allows for a maximum clock frequency of 50MHz. [11] The clock signal in Figure 7 faces similar distortion issues to the 42MHz clock from the microcontroller but at a greater degree due to being a higher frequency. Since none of these effects occur at lower frequencies, this appears to be a limitation of the probe being used, not the board.

3.1.3 Final Choice

Both hardware choices display similar results in testing with clock signals meaning they are both capable of being used to develop a BERT. The FPGA has a higher maximum clock frequency (50MHz) over the microcontroller (42MHz) suggesting it can be pushed further to test higher bitrates. [7, 8, 11]
Microcontrollers are programmed at a higher level than FPGAs, meaning they are easier to work with. However, this provides minimal control over the lower level logic. FPGAs allow for a higher degree of control by working directly at the logic circuit level, but this makes them difficult to program.
Microcontrollers have interfaces for transmitting data synchronised to a clock e.g. I2C, SPI. But these peripherals have complex workings which may require more than one clock cycle per transferred bit. The FPGA being used has an ADC but no other peripherals since it works directly at the logic circuit level. Despite being difficult to program, logic circuits can be designed in a way that allows for one bit to be transferred per clock cycle. [8, 11] When working with registers, a low level of programming is required for direct control. Microcontrollers tend to be cheaper than FPGAs, and low cost is one of the aims of this project. However, compared to the price of BER equipment, this difference in price is small, meaning despite the choice of hardware, the costs will most likely be relatively low. [2, 5]
In this project, an LFSR would be required for PRBS generation, meaning direct control over logic is necessary. The clock rates available are in the Megahertz range rather than Gigahertz, so it would be ideal use clock cycles efficiently. Considering these factors, the FPGA has been chosen to develop the BERT.

3.2 PRBS generator

For this project, a PRBS generator will be used to generate a bitstream for the BERT. At this point in time, a PRBS-7 generator has been made. It repeats every 127 bits and is represented by the following polynomial. [3]

$$x^7 + x^6 + 1 \tag{1}$$

This polynomial isn't mathematical, it's a representation of the LFSR that would generate a PRBS-7. The values of the powers represent which bits are used for feedback. This polynomial says that the 7th and 6th bits of the register are inputs to an XOR gate where the output feeds into the Least Significant Bit of the LFSR. The Most Significant Bit of the LFSR is transmitted to the communication channel on each clock cycle. [3, 4] This can be implemented using D-Type Flip-Flops in Quartus Prime. Typically, larger LFSRs would be used for PRBS generation to achieve a higher degree of randomness, as demonstrated in the Serial Communication BERT mentioned previously. [2]

3.2.1 Push Button Clock

For testing purposes, the clock will be controlled by a push button on the DE0-Nano with the status of the LFSR on LEDs. Refer to Figure 18 in the appendices for the logic circuit. The initial status of the LFSR is set to 7 ones. The buttons on the FPGA are active low meaning they become logic 0 when pressed and logic 1 when released. The LFSR status changes when the button is released, not when pressed down, meaning the PRBS generator is synchronised to the clock's rising edge which is expected. Once the button is pressed 127 times, the LFSR's status returns to the original state of 7 ones. Hence, PRBS generator produces a repetitive bit sequence of length 127, which is also expected. When the initial seed is 7 ones, the expected repeating 127 bit sequence is the following.
1111111000000100000110000101000111100100010110011101010011111101000011100
0100100110110101101111011000110100101110111001100101010
This can be compared to the output of the PRBS generator viewed on an oscilloscope.

3.2.2 500kHz Clock

The 50MHz oscillator on the FPGA is initiated and connected to a Phase Locked Loop (PLL) to divide it by 100, producing a 500kHz clock. This clock controls the PRBS generator. Refer to Figure 19 in the appendices for the logic circuit.

Figure 8: Oscilloscope capture of PRBS generator at 500kHz (first 60 bits)

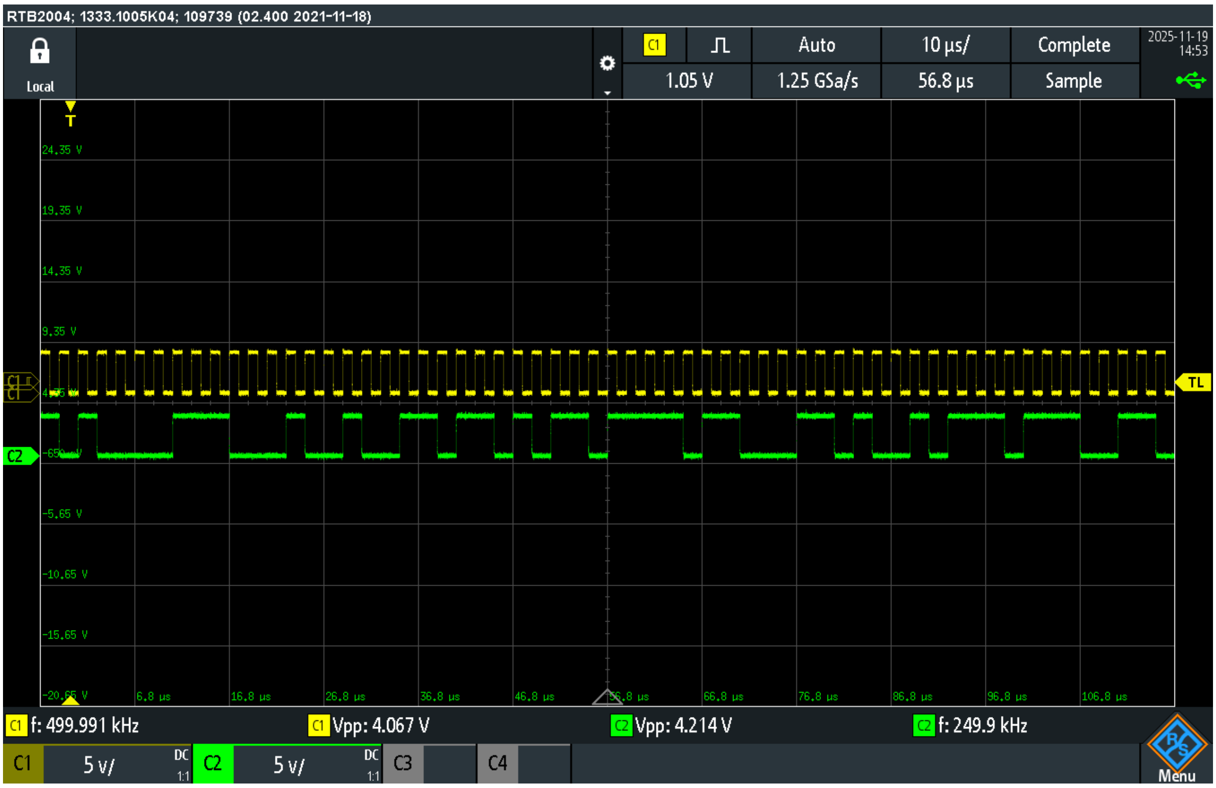


Figure 9: Oscilloscope capture of PRBS generator at 500kHz (61^{st}-120^{th} bits)

Figure 10: Oscilloscope capture of PRBS generator at 500kHz. First 7 bits in the figure are the 121^{st}-127^{th} bits of the sequence. Sequence repeats after

In Figure 8, Figure 9 and Figure 10 channel 1 (CH1) is the 500kHz clock and channel 2 (CH2) is the PRBS generator output. At 500kHz, the probe doesn't distort the square waveforms. When comparing the output to the expected bit sequence, no mistakes are found and the sequence repeats every 127 bits as expected.

3.2.3 10MHz Clock

The 50MHz oscillator on the FPGA is initiated and connected to a PLL to divide it by 5, producing a 10MHz clock. This clock controls the PRBS generator. Refer to Figure 20 in the appendices for the logic circuit.

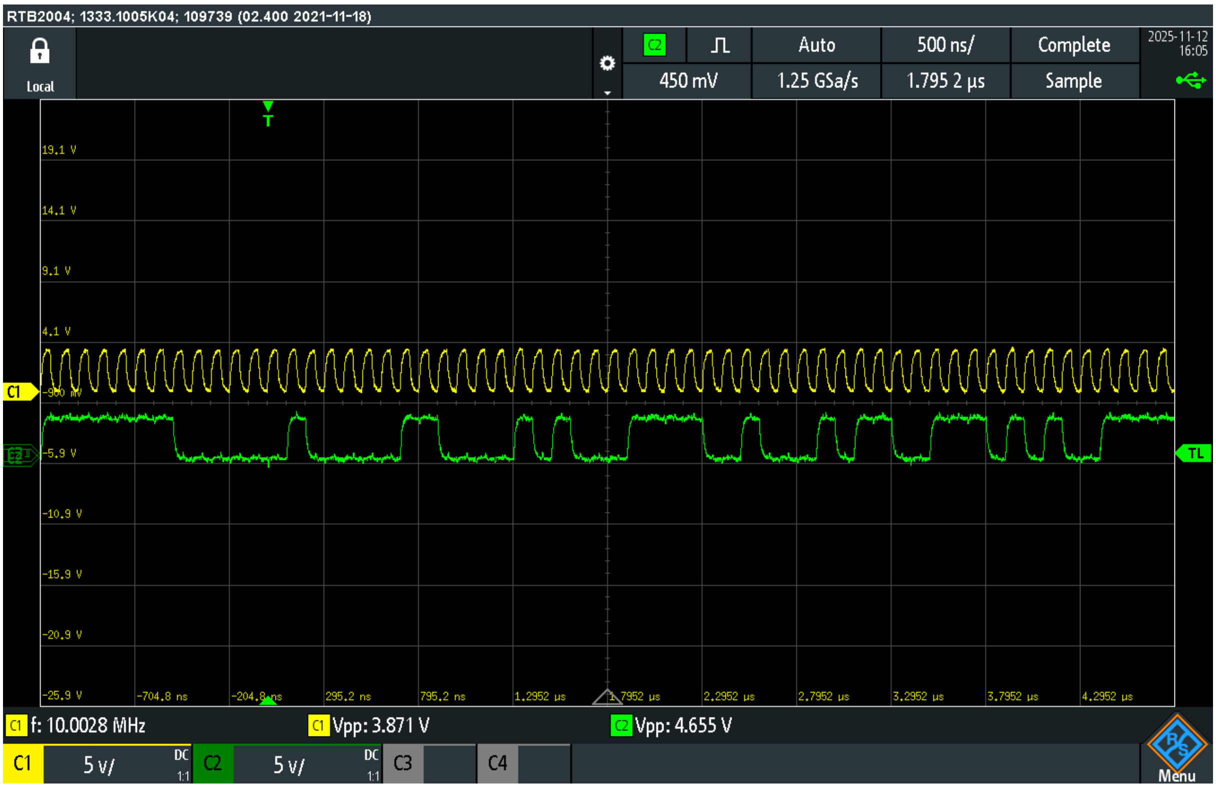


Figure 11: Oscilloscope capture of PRBS generator at 10MHz (first 60 bits)

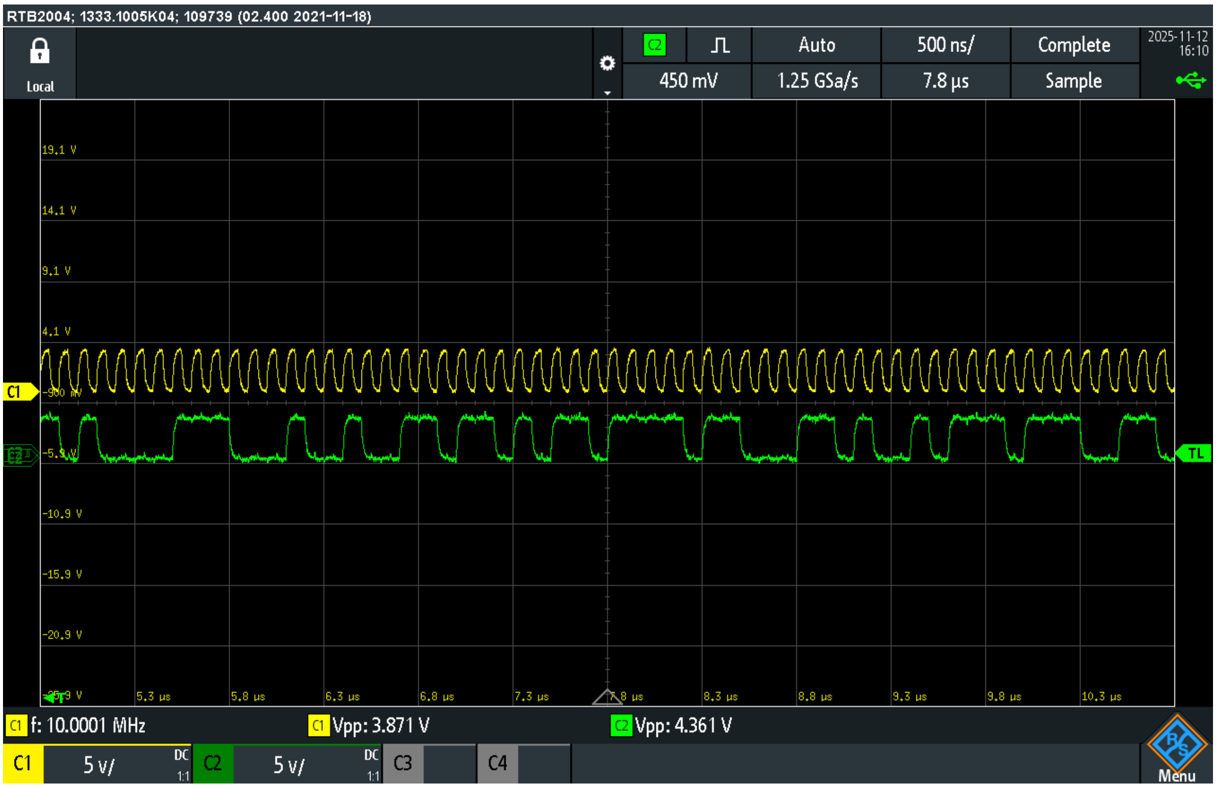


Figure 12: Oscilloscope capture of PRBS generator at 10MHz (61st-120th bits)

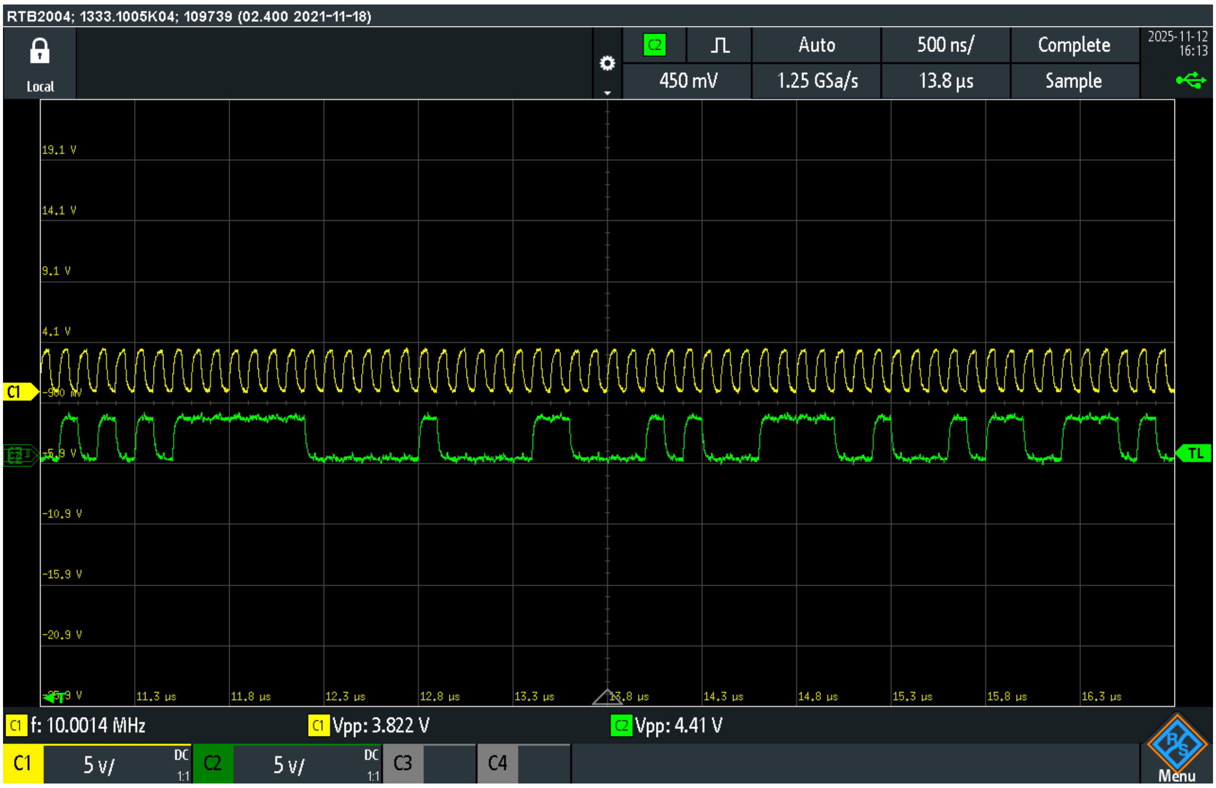


Figure 13: Oscilloscope capture of PRBS generator at 10MHz. First 7 bits in the figure are the 121st-127th bits of the sequence. Sequence repeats after

In Figure 11, Figure 12 and Figure 13, CH1 is the 10MHz clock and CH2 is the PRBS generator output. At 10MHz, the probes distorts the square waveform due to high frequency components of the wave being attenuated. The probe used has a -3dB bandwidth of 10MHz. [9, 10] No mistakes are found when comparing the output to the expected bit sequence and the sequence repeats every 127 bits as expected.

3.2.4 20MHz Clock

The PLL to multiply the 50MHz oscillator by 2/5, producing a 20MHz clock. This clock controls the PRBS generator. Refer to Figure 21 in the appendices for the logic circuit.

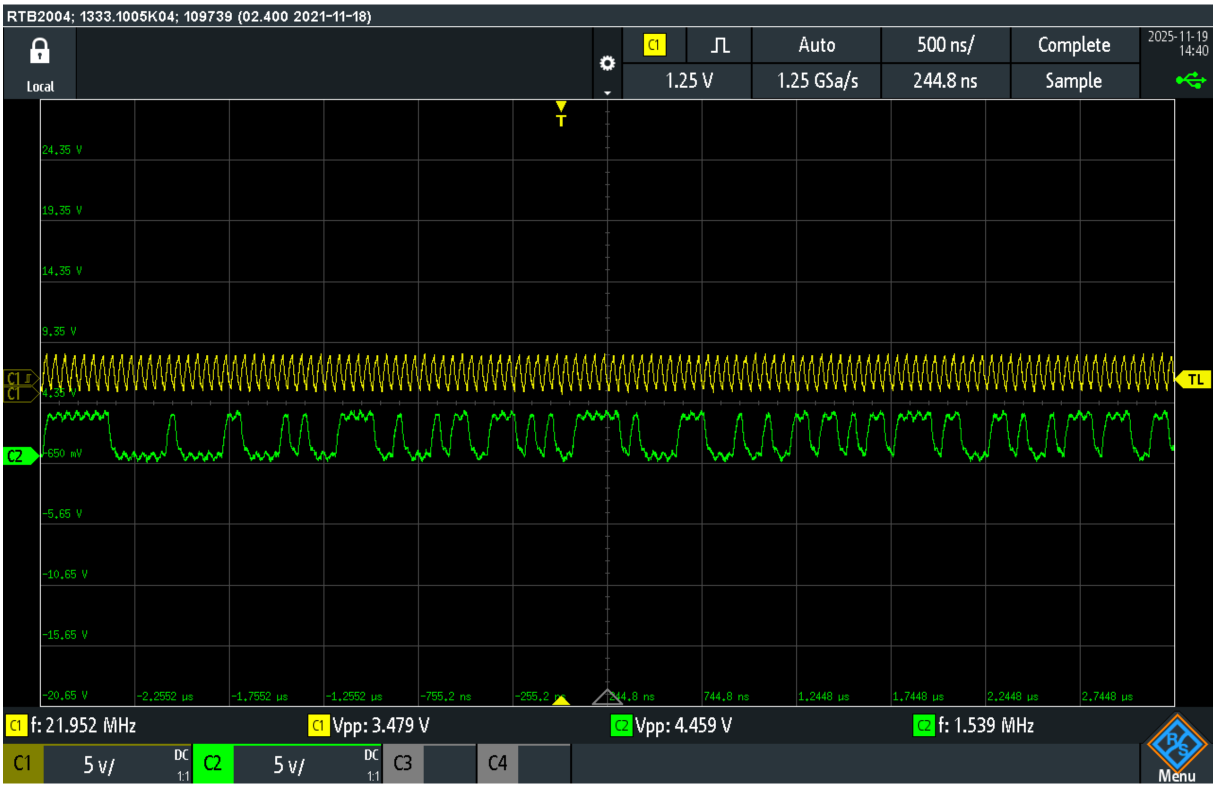


Figure 14: Oscilloscope capture of PRBS generator at 20MHz (first 120 bits)

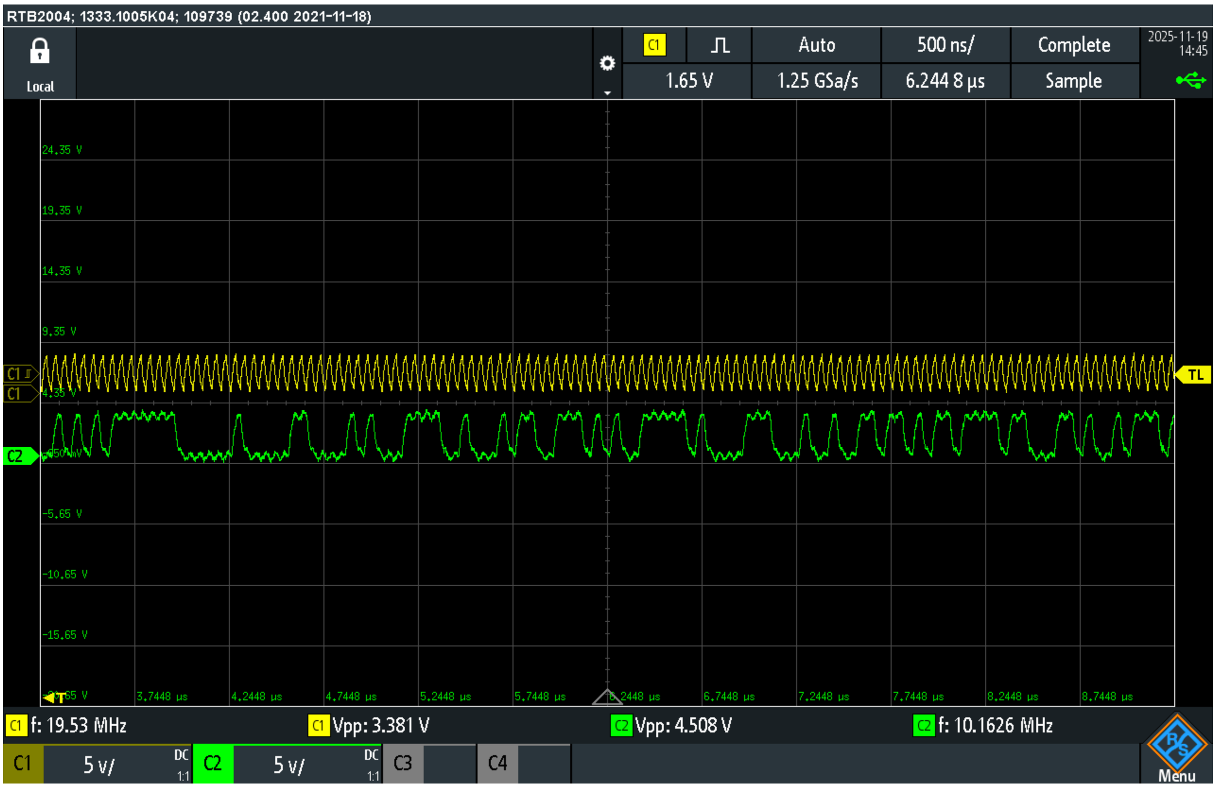


Figure 15: Oscilloscope capture of PRBS generator at 20MHz First 7 bits in the figure are the 121st-127th bits of the sequence. Sequence repeats after

In Figure 14 and Figure 15, CH1 is the 20MHz clock and CH2 is the PRBS generator output. At 20MHz, the probe distorts the square wave due to high frequency components of the wave being attenuated. This happens at a greater degree than at 10MHz. [10] The bit sequence produced at 20MHz is the same as the expected bit sequence.

4 Project Management

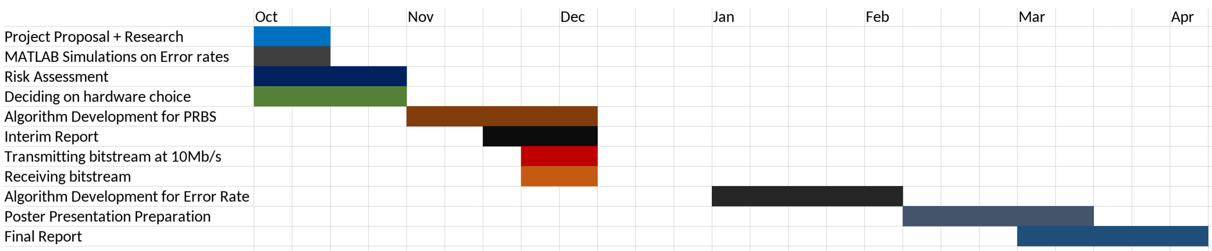


Figure 16: Original Gantt Chart

So far, the tasks have been completed on schedule with tasks like the risk assessment and hardware choice being completed quicker than expected. There was an overlap in time in hardware decision and PRBS development since generating a clock signal is a part of it.

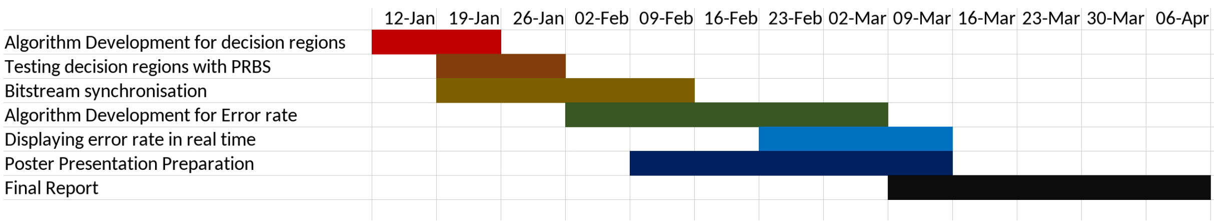


Figure 17: Modified Gantt Chart for Term 2

The tasks for Term 2 have been broken down further. Algorithm development for decision regions isn't a priority and will be disregarded if reasonable progress isn't made. Bitstream synchronisation must be completed first since it's a required condition for calculating the BER. This part may be the most difficult hence it has the most allocated time.

5 Future Work

For the decision regions, a voltage threshold will be used where above would be considered logic 1 and below would be considered logic 0. This would require using an ADC to read voltages. In bitstream synchronisation, a method of aligning the beginning of the received and generated PRBS needs to be developed. For BER calculations, a fixed number of bits would need to be set for which the received and generated bitstreams are compared for to display the BER as a value.

References

[1] S. M. J. Alam, M. R. Alam, G. Hu, and M. Z. Mehrab, "Bit Error Rate Optimization in Fiber Optic Communications," *International Journal of Machine Learning and Computing*, vol. 1, no. 5, pp. 435–440, Dec. 2011, doi:https://doi.org/10.7763/IJMLC.2011.V1.65.

[2] M. J. Basford, A. E. Pena-Quintal, S. Greedy, M. Sumner, and D. W. P. Thomas, "An Open Source, FPGA-Based Bit Error Ratio Tester for Serial Communications," in *2020 International Symposium on Electromagnetic Compatibility - EMC EUROPE*, Rome, Italy, Sep. 2020, pp. 1–6, doi:https://doi.org/10.1109/EMCEUROPE48519.2020.9245786.

[3] D. Riccardi and P. Novellini, "An Attribute-Programmable PRBS Generator and Checker," AMD, Santa Clara, California, Tech. Rep. v1, 2011, Available: https://docs.amd.com/v/u/en-US/xapp884_PRBS_GeneratorChecker, (Accessed Dec. 05 2025).

[4] M. George and P. Alfke, "Linear Feedback Shift Registers in Virtex Devices," AMD, Santa Clara, California, Tech. Rep. v1.3, 2007, Available: https://docs.amd.com/v/u/en-US/xapp210, (Accessed Dec. 05 2025).

[5] S. Chatterjee and B. Tiru, "Design and testing of a bit error rate tester with application to a visible light communication system," *Materials Today: Proceedings, (In Press)*, Apr. 2023, doi:https://doi.org/10.1016/j.matpr.2023.03.786.

[6] N. Marda, G. Vaishnav, and U. S. N. Rao, "Bit Error Rate Testing Scheme for Digital Communication Devices," in *The 2014 International Conference on Control, Instrumentation, Energy and Communication (CIEC)*, Calcutta, India, Jan. 2014, pp. 490–493, doi:https://doi.org/10.1109/CIEC.2014.6959137.

[7] STMicroelectronics, "NUCLEO-xxxxCx, NUCLEO-xxxxRx, NUCLEO-xxxxRx-P, NUCLEO-xxxxRx-Q Data brief," Datasheet for NUCLEO-F401RE board, Feb. 2014, [Revised Jan. 2025]. Available: https://www.st.com/resource/en/data_brief/nucleo-f401re.pdf, (Accessed Dec. 05 2025).

[8] ——, "STM32F401xD STM32F401xE," Datasheet for STM32F401RE Microcontroller, Jan. 2014, "[Revised Feb. 2014] Available: https://www.st.com/resource/en/datasheet/stm32f401re.pdf, (Accessed Dec. 05 2025)".

[9] Rohde and Schwartz, "R&S RT-ZP03 Manual," User Manual for RT-ZPO3 Oscilloscope Probe, May 2019, "[Revised May 2019] Available: https://www.rohde-schwarz.com/ae/manual/r-s-rt-zp03-manual-manuals_78701-172932.html, (Accessed Dec. 05 2025)".

[10] J. A. Svoboda and R. C. Dorf, "Fourier Series and Fourier Transform," in *Introduction to Electric Circuits*, 9th ed., Daniel Sayre and Wendy Ashenberg, Ed. New Jersey: Hoboken: John Wiley & Sons, 2013, pp. 741–802.

[11] TerasicTechnologies, "DE0-Nano User Manual," Datasheet for DE0-Nano EP4CE22F17C6, Jan. 2011, "[Revised Mar. 2021] Available: https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=139&No=593&PartNo=4#contents, (Accessed Dec. 05 2025)".

Appendices

A Appendices

1.1 Quartus Prime Lite Block Diagrams

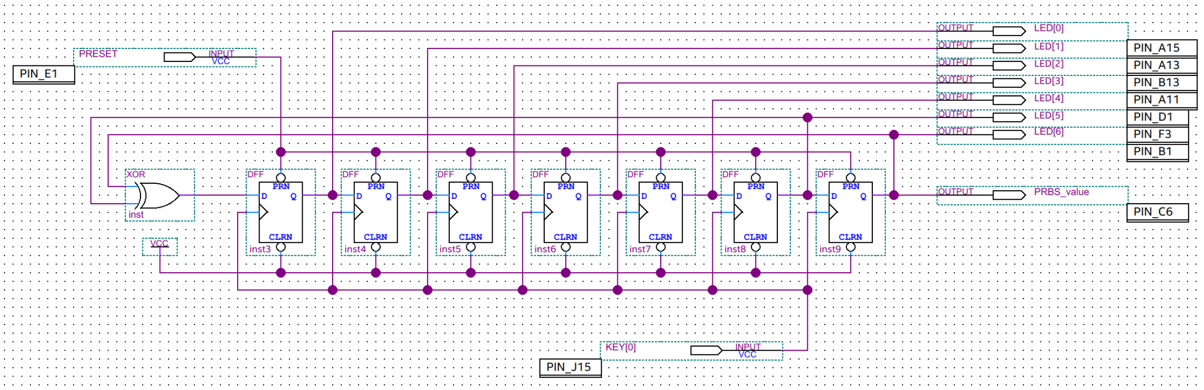


Figure 18: PRBS Generator with the clock tied to push button

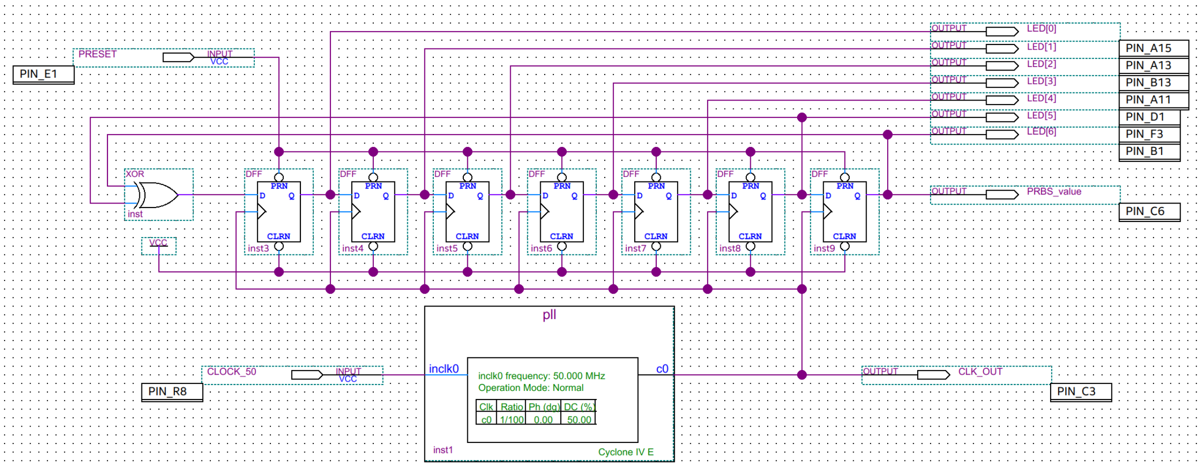


Figure 19: PRBS Generator implemented with 500kHz clock

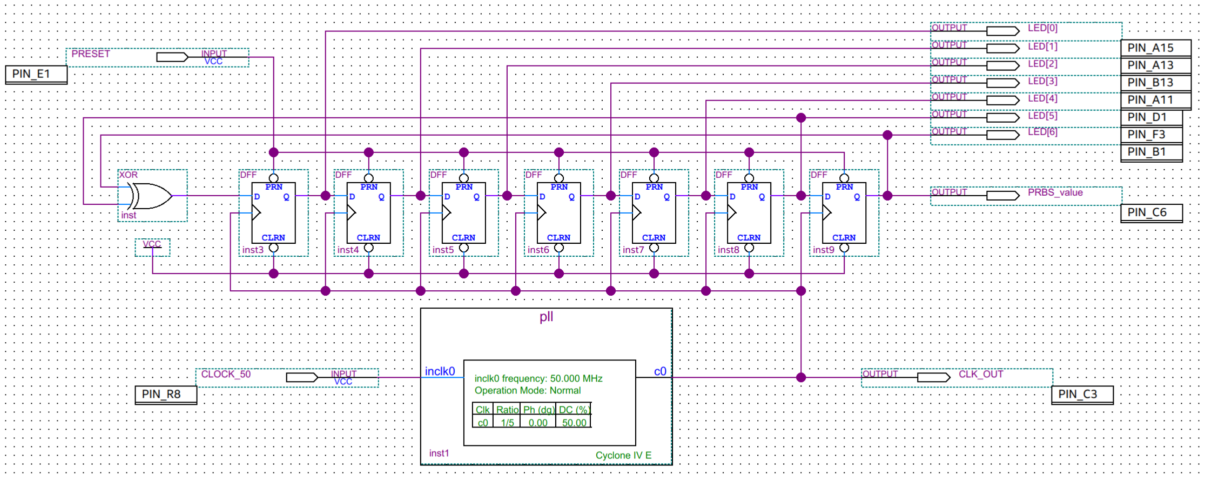


Figure 20: PRBS Generator implemented with 10MHz clock

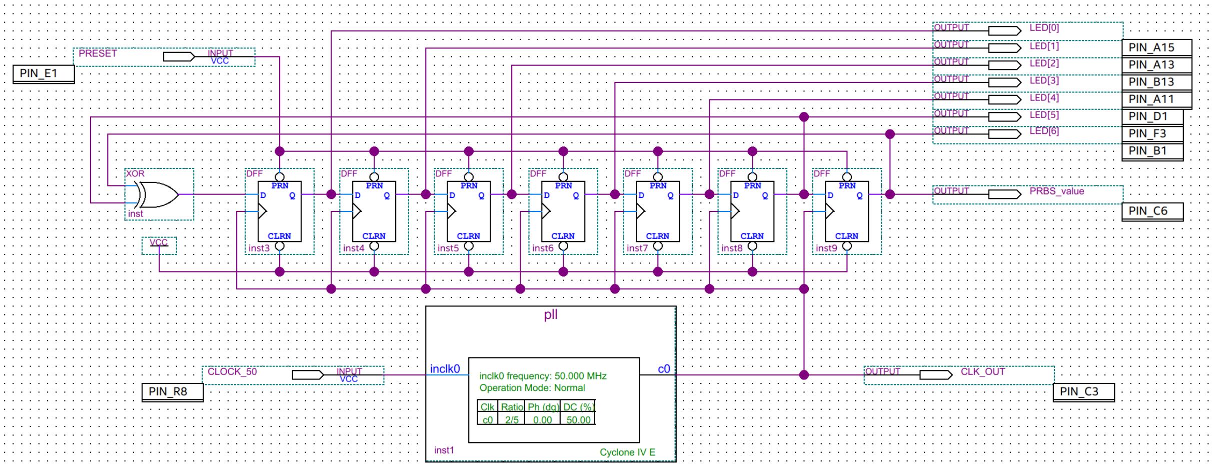


Figure 21: PRBS Generator implemented with 20MHz clock